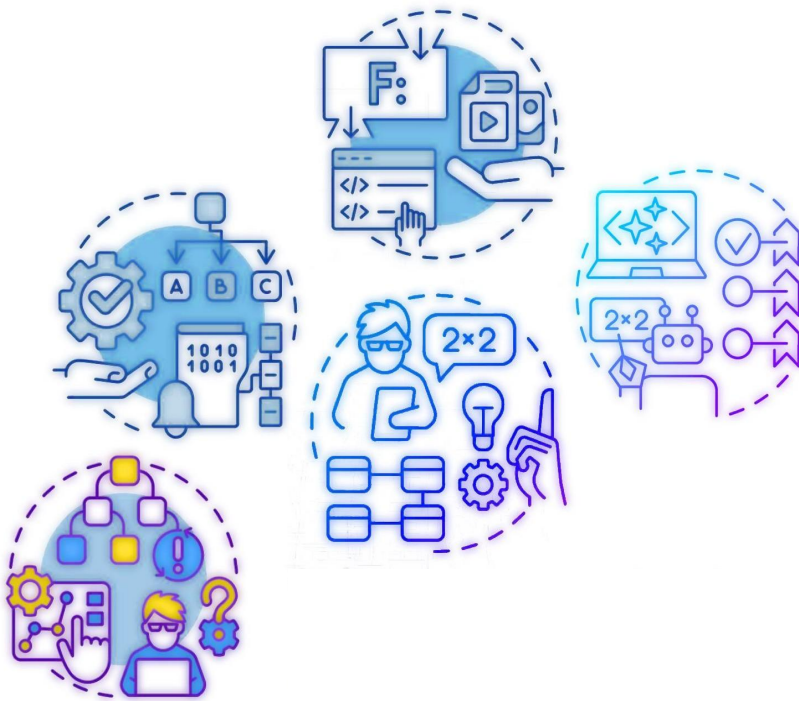




Paradigma Lógico

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. J. Exequiel Benavidez
Ing. Jimena Bourlot

Tecnologías de la Programación

1 – EL CORTE

- Prolog realiza backtracking automático para resolver objetivos.
- El programador queda librado de tener que programarlo en forma explícita.
- Pero hay situaciones en las cuales para mejorar la eficiencia queremos prevenir efectos causados por backtracking
- Para controlar la búsqueda se utiliza el símbolo “!”, el cual aparece como un predicado más.
- “!” siempre tiene éxito y provoca el descarte de todas las alternativas (ramas) que quedaron pendientes de ser exploradas desde el Instante en que se utilizó la regla que contiene dicho “!”

1 – EL CORTE

Supongamos las siguientes reglas:

- R1: Si $X < 3$ entonces $Y = 0$
- R2: Si $3 \leq X < 6$ entonces $Y = 2$
- R2: Si $6 \leq X$ entonces $Y = 4$

En Prolog:

- `f(X, 0) : write('Regla 1'), X < 3.`
- `f(X, 2) : write('Regla 2'), X >= 3, X < 6.`
- `f(X, 4) : write('Regla 3'), X >= 6.`

1 – EL CORTE

Funcionamiento de cada regla:

1. **$f(X, 0) :- \text{write('Regla 1')}, X < 3, !.$**

- Significado: Si se le pasa un valor X a f , y X es menor que 3, entonces este predicado tendrá éxito y el segundo argumento será 0.
- Comportamiento:
 - Imprime el texto 'Regla 1'.
 - Verifica la condición $X < 3$.
 - Si la condición es verdadera, el ! (corte) se activa. Esto significa que Prolog no intentará las reglas siguientes para $f/2$ (ni $f(X, 2)$ ni $f(X, 4)$) para este intento de solución. La solución $Y=0$ es la única que se encontrará.

1 – EL CORTE

Funcionamiento de cada regla:

2. $f(X, 2) :- \text{write('Regla 2')}, X \geq 3, X < 6, !.$

- Significado: Si la primera regla falló (porque X no era menor que 3), y ahora X es mayor o igual que 3 Y menor que 6, entonces este predicado tendrá éxito y el segundo argumento será 2.
- Comportamiento:
 - Imprime el texto 'Regla 2'.
 - Verifica las condiciones $X \geq 3$ y $X < 6$.
 - Si ambas son verdaderas, el ! (corte) se activa. Prolog no intentará la última regla ($f(X, 4)$) para este intento.

1 – EL CORTE

Funcionamiento de cada regla:

3. $f(X, 4) :- \text{write('Regla 3')}, X \geq 6, !.$

- Significado: Si las dos reglas anteriores fallaron (porque X no era menor que 3, ni estaba entre 3 y 6), y ahora X es mayor o igual que 6, entonces este predicado tendrá éxito y el segundo argumento será 4.
- Comportamiento:
 - Imprime el texto 'Regla 3'.
 - Verifica la condición $X \geq 6$.
 - Si es verdadera, el ! (corte) se activa. Como es la última regla, el corte aquí principalmente evita que Prolog busque otras soluciones para esta misma regla (aunque en este caso no hay más opciones).

1 – EL CORTE

Funcionamiento de cada regla:

3. `f(X, 4) :- write('Regla 3'), X >= 6, !.`

- Significado: Si las dos reglas anteriores fallaron (porque X no era menor que 3, ni estaba entre 3 y 6), y ahora X es mayor o igual que 6, entonces este predicado tendrá éxito y el segundo argumento será 4.
- Comportamiento:
 - Imprime el texto 'Regla 3'.
 - Verifica la condición $X \geq 6$.
 - Si es verdadera, el ! (corte) se activa. Como es la última regla, el corte aquí principalmente evita que Prolog busque otras soluciones para esta misma regla (aunque en este caso no hay más opciones).

1 – EL CORTE

En resumen, estas reglas implementan una lógica de "si-entonces-sino-si-entonces-sino-entonces" (if-then-else if-else):

- Si $X < 3$, entonces el resultado es 0 y se aplica la Regla 1.
- Sino, si $3 \leq X < 6$, entonces el resultado es 2 y se aplica la Regla 2.
- Sino (si $X \geq 6$), entonces el resultado es 4 y se aplica la Regla 3.

El corte (!) garantiza que solo una de estas reglas se ejecutará para un valor dado de X , haciendo que el comportamiento sea predecible y que la búsqueda de soluciones sea eficiente.

1 – EL CORTE

Si consultáramos:

- $f(1, Y)$, $2 < Y$.

El resultado sería:

- Regla 1 Regla 2 Regla 3
fail.

Como se ve, trata de unificar las tres reglas, por más que sepamos son excluyentes según el valor del primer argumento.

1 – EL CORTE

Una solución más eficiente a la primera podría ser:

- $f(X, 0) : \text{write('Regla 1')}, X < 3, !.$
- $f(X, 2) : \text{write('Regla 2')}, X \geq 3, X < 6, !.$
- $f(X, 4) : \text{write('Regla 3')}, X \geq 6, !.$

El resultado ahora sería:

- Regla 1
fail.

1 – EL CORTE

Ahora que eliminamos el backtracking podemos reescribir las reglas de la siguiente forma:

$f(X, 0) : \text{write('Regla 1')}, X < 3, !.$

$f(X, 2) : \text{write('Regla 2')}, X < 6, !.$

$f(X, 4) : \text{write('Regla 3')}.$

El punto a tener en cuenta en ésta última solución, es que se está cambiando el significado declarativo de las reglas.

1 – EL CORTE

Diferencia entre el significado declarativo y el significado procedural de un programa.

Significado Declarativo (¿Qué es verdad?): Se refiere a la lógica pura del programa, es decir, qué hechos y relaciones son verdaderos según las reglas y los hechos definidos, sin importar cómo Prolog los encuentra. Es la "especificación" lógica del problema.

Significado Procedural (¿Cómo se encuentra la verdad?): Se refiere al proceso de ejecución de Prolog, cómo busca soluciones, el orden de las cláusulas, el efecto del backtracking y, crucialmente, el impacto del corte (!).

1 – EL CORTE

Los cortes se usan normalmente para:

- No buscar soluciones en predicados alternativos
En éste caso se sabe que se ha escogido la regla adecuada para el objetivo y no tiene sentido seguir buscando otra, no la hay.
- Combinación de corte y fallo.
- Solo es necesaria la primera opción
En éste caso se sabe que se ha llegado a la única solución, o solo se necesita alguna, en cualquier caso no tiene sentido seguir buscando otras

1 – EL CORTE

Combinación de corte y fallo

- Nos permite indicarle a Prolog que falle inmediatamente sin intentar encontrar soluciones alternativas a la regla.
- La combinación de corte y fallo definen la negación (not) en prolog:
 - `not(P): -P, !, fail.`
 - `not(P).`
- Tener en cuenta que la negación “not” definida de esta forma siempre lleva implícito el corte.

1 – EL CORTE

Combinación de corte y fallo

- Por legibilidad conviene usar la negación como fallo (not), en vez del corte-fallo. Siempre es posible sustituir el corte/fallo por el not.
- Supongamos que necesitamos calcular la dosis de remedio a indicarle a una persona según su peso, podríamos indagar por: dosis(juan, 70, Dosis) y para indicarle el remedio a una persona, ésta debe cumplir ciertas condiciones, podríamos escribir:

1 – EL CORTE

- Combinación de corte y fallo
dosis(Persona, _, _) : - enfermo_renal(Persona), !, fail.
dosis(Persona, _, _) : - con_ulcera_gastrica(Persona), !, fail.
dosis(Persona, _, _) :
 hipersensible_bromexhina(Persona), !, fail.
 dosis(Persona, Peso, Dosis) : - Dosis is Peso * 2.
- O utilizando la negación:
dosis(Persona, Peso, Dosis) :-
 not(enfermo_renal(Persona)),
 not(con_ulcera_gastrica(Persona)),
 not(hipersensible_bromexhina(Persona)),
 Dosis is Peso * 2.

1 – EL CORTE

Problemas

Cortes verdes:

No afectan el significado declarativo de los programas.

Los programas se pueden leer sin tener en cuenta los cortes

Cortes rojos:

Afectan el significado declarativo de los programas.

Es más fácil cometer errores, se debe prestar especial atención al significado procedural en los mismos.

1 – EL CORTE

Ejemplo

$p:- a, b. \text{ \% } p \Leftarrow (a \wedge b) \vee c$

$p:- c.$

El siguiente corte es rojo y cambia el significado

$p:- a, !, b. \text{ \% } p \Leftarrow (a \wedge b) \vee (\neg a \wedge c)$

$p:- c.$

Invirtiendo los términos

$p:- c. \text{ \% } p \Leftarrow c \vee (a \wedge b)$

$p:- a, !, b.$

2 – EL ESQUEMA CONDICIONAL

En Prolog la conjunción se simboliza con la coma (“,”) y la disyunción mediante la escritura de más de una regla para la misma cabecera.

Otra forma de implementar la disyunción es mediante el punto y coma (“;”)

- $p \text{ :- } a, b.$
- $p \text{ :- } a, c.$
- $P \text{ :- } d.$
- $p \text{ : } a, b; a, c; d.$

2 – EL ESQUEMA CONDICIONAL

Por cuestiones de legibilidad, se recomienda usar varias cláusulas para el mismo predicado. Si se usa la notación de la derecha, es altamente recomendable separar criteriosamente los bloques en líneas distintas y usar paréntesis para mejorar la lectura.

```
p :- (a, b);  
      (a, c);  
      d.
```

2 – EL ESQUEMA CONDICIONAL

La cláusula condicional if-then-else se representa en prolog como “if > then; else”

- `integer(1) > write('es entero'); write('no es entero').`
 > es entero
- `integer(1.1) > write('es entero'); write('no es entero').`
 > no es entero